

messages: 4

## 強化學習環境邏輯實作

You

資料格式：  
Datetime,NVDA\_Close,NVDA\_High,NVDA\_Low,NVDA\_Open,NVDA\_Volume,AAPL\_Close,AAPL\_High,AAPL\_Low,AAPL\_Open,AAPL\_Volume,JPM\_Close,JPM\_High,JP  
2026-01-02  
09:00:00+00:00,190.15,190.25,188.61,189.24,0.0,273.69,274.0,272.99,273.11,0.0,323.59,323.85,322.25,322.25,0.0,113.79,113.99,113.62,113.77,0.0,120.6,120.85,120.3  
2026-01-02  
10:00:00+00:00,190.01,190.27,190.0,190.1,0.0,273.24,273.74,273.2,273.66,0.0,323.6,324.0,323.32,323.59,0.0,113.77,113.86,113.71,113.82,0.0,120.6,120.72,120.45,120  
環境設定： "" state = [Close,High,Low,Open,Volume,shares,cash] initial shares = [0,0], initial cash = 1000, old\_value = initial cash transaction\_rate = 0.003 #0.3%  
的手續費

action = [share1,share2,cash] share\_rate 範圍落在0~1，表示資產配置的比例 對action 做softmax 使總值保持在1 new\_action = softmax(action) new\_action = [share\_rate1,share\_rate2,cash\_rate]

計算總資產

```
portfolio_value = 0  
  
for i in range(len(shares)): portfolio_value += shares[i] * current_price[i]  
  
portfolio_value += cash
```

計算舊權重

```
for i in range(len(shares)): old_share_rate[i] = (shares[i] * current_price[i]) / portfolio_value  
  
old_cash_rate = cash / portfolio_value
```

計算差值(比例)

```
for i in range(len(shares)): delta_share[i] = share_rate[i] - old_share_rate[i]  
  
delta_cash = cash_rate - old_cash_rate
```

算出實際交易金額

```
for i in range(len(shares)): trade_value[i] = delta_share[i] * portfolio_value
```

買賣動作(以第一支股票為例)

```
price[0] = next_open_price[0]  
  
if trade_value[0] > 0: buy_shares = trade_value[0] / price[0] fee = trade_value[0] * transaction_rate  
cash -= trade_value[0] + fee shares[0] += buy_shares  
  
else: sell_shares = abs(trade_value[0]) / price[0] fee = abs(trade_value[0]) * transaction_rate  
cash += abs(trade_value[0]) - fee shares[0] -= sell_shares  
  
action 之後計算value  
  
value = money + shares * current_price  
  
reward: return (value - old_value)/old_value  
  
重置old_value old_value = value "" 資料是股市的OHLCV，根據環境設定寫一個Q-learning的程式
```

Gemini

根據你提供的資料格式與交易邏輯，這是一個典型的連續動作空間 (Continuous Action Space) 投資組合再平衡問題。

在強化學習中，傳統的 **Q-Learning** 通常用於處理「離散動作」（例如：買、賣、觀望）。然而，你的環境設定使用了 softmax 來決定資產配置比例，這通常對應到 **深度 Q 網路 (DQN)** 的變體，或是更適合處理連續空間的演算法。

為了符合你的需求，我將程式碼分為兩個部分：

- 1. **TradingEnv** 類別：完全依照你提供的邏輯（手續費、Softmax 權重、資產計算）封裝成一個模擬環境。
- 2. **DQN** 代理人：實作深度 Q 學習的核心邏輯。

1. 交易環境實作 (Python)

```
import numpy as np  
import pandas as pd  
  
class TradingEnv:  
    def __init__(self, df, initial_cash=1000, transaction_rate=0.003):  
        self.df = df  
        self.initial_cash = initial_cash  
        self.transaction_rate = transaction_rate  
        self.reset()  
  
    def reset(self):  
        self.current_step = 0  
        self.cash = self.initial_cash
```

```

self.shares = np.array([0.0, 0.0]) # 假設處理前兩支股票: NVDA, AAPL
self.portfolio_value = self.initial_cash
self.old_value = self.initial_cash
return self._get_observation()

def _get_observation(self):
    # 狀態包含: OHLCV (當前列) + 目前持有股數 + 目前現金
    obs = self.df.iloc[self.current_step, 1:].values.tolist()
    obs.extend(self.shares.tolist())
    obs.append(self.cash)
    return np.array(obs)

def _softmax(self, x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()

def step(self, action):
    # 1. Softmax 轉換動作為比例 [share_rate1, share_rate2, cash_rate]
    new_action = self._softmax(action)
    share_rates = new_action[:2]
    cash_rate = new_action[2]

    # 2. 取得當前價格 (Close) 與 下一期開盤價 (用於交易)
    # 這裡假設 current_price 用於計算當前價值, next_open 用於執行交易
    current_prices = np.array([self.df.iloc[self.current_step]['NVDA_Close'],
                               self.df.iloc[self.current_step]['AAPL_Close']])

    # 檢查是否還有下一行資料
    if self.current_step >= len(self.df) - 1:
        return self._get_observation(), 0, True

    next_open_prices = np.array([self.df.iloc[self.current_step + 1]['NVDA_Open'],
                                 self.df.iloc[self.current_step + 1]['AAPL_Open']])

    # 3. 計算總資產 (Portfolio Value)
    self.portfolio_value = np.sum(self.shares * current_prices) + self.cash

    # 4. 計算舊權重與差值
    old_share_rates = (self.shares * current_prices) / self.portfolio_value
    delta_shares = share_rates - old_share_rates

    # 5. 執行交易 (依照你提供的邏輯)
    for i in range(len(self.shares)):
        trade_value = delta_shares[i] * self.portfolio_value
        price = next_open_prices[i]

        if trade_value > 0: # 買入
            buy_shares = trade_value / price
            fee = trade_value * self.transaction_rate
            self.cash -= (trade_value + fee)
            self.shares[i] += buy_shares
        else: # 賣出
            sell_shares = abs(trade_value) / price
            fee = abs(trade_value) * self.transaction_rate
            self.cash += (abs(trade_value) - fee)
            self.shares[i] -= sell_shares

    # 6. 計算新價值與 Reward
    # 交易後的價值更新 (以交易當下的價格估算)
    new_value = np.sum(self.shares * next_open_prices) + self.cash
    reward = (new_value - self.old_value) / self.old_value

    self.old_value = new_value
    self.current_step += 1
    done = self.current_step >= len(self.df) - 1

    return self._get_observation(), reward, done

```

---

## 2. Q-Learning (DQN) 核心邏輯

由於狀態空間是連續的 (價格與交易量), 我們使用神經網路來近似 Q 函數。

```

import torch
import torch.nn as nn
import torch.optim as optim
import random

class QNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(QNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, action_dim)
        )

    def forward(self, x):
        return self.fc(x)

# 訓練迴圈簡化版
def train_dqn(env, episodes=100):

```

```

state_dim = 28 # 根據你的資料欄位計算 (OHLCV*5 + shares*2 + cash)
action_dim = 3 # [share1_weight, share2_weight, cash_weight]

model = QNetwork(state_dim, action_dim)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.MSELoss()

for ep in range(epochs):
    state = env.reset()
    done = False
    total_reward = 0

    while not done:
        state_tensor = torch.FloatTensor(state).unsqueeze(0)

        # Epsilon-greedy 或直接輸出 Action
        # 在你的環境中，model 輸出的 raw values 會被 env 裡的 softmax 處理
        action_values = model(state_tensor)
        action = action_values.detach().numpy()[0]

        next_state, reward, done = env.step(action)

        # 計算回報 (簡化版 Q-learning 更新)
        target = reward + 0.99 * torch.max(model(torch.FloatTensor(next_state).unsqueeze(0))).item()
        loss = criterion(action_values, torch.FloatTensor([target] * action_dim)) # 簡化標籤

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        state = next_state
        total_reward += reward

    print(f"Episode: {ep}, Total Reward: {total_reward:.4f}")

```

## 關鍵公式說明

在你的環境中，獎勵函數 (Reward) 定義為資產變動率：

$$Reward = \frac{Value_{new} - Value_{old}}{Value_{old}}$$

這是一個非常直觀的信號，能引導 Agent 學習如何極大化資產成長率。

## 實作建議

- 資料預處理**：建議對 NVDA\_Volume 等數值進行歸一化 (Scaling)，因為原始交易量與股價的數值差異極大，會導致神經網路收斂緩慢。
- Softmax 的角色**：在 DQN 中，通常 action 是離散的索引。但因為你要求使用 `new_action = softmax(action)`，這意味著你的網路輸出層實際上是在預測各資產的最佳比例分配。
- 手續費懲罰**：你的邏輯中包含了 `transaction_rate`。這非常重要，因為它會阻止 Agent 進行無意義的頻繁換倉 (Churning)，Agent 必須確保預期收益大於 0.3% 的成本才會執行大動作。

這套程式碼可以直接將你的 CSV 數據餵入並開始模擬交易流程。你需要根據實際股票數量調整 `self.shares` 的維度與 `action_dim` 的大小。